Week-1

Week-1

🔷 1. Software Testing kya hota hai?

Software Testing ka matlab hota hai:

> Software ko check karna ki wo **sahi kaam kar raha hai ya nahi**, aur **requirements ke according hai ya nahi**.

Simple words me:

- Jo software banaya → usme **bug/error** hai ya nahi
- User jo expect kar raha → software wahi deliver kar raha ya nahi

Testing ka goal **bug dhoondhna** hota hai, ye prove karna nahi ki software perfect hai.

🔷 2. Verification vs Validation (Very Important 🔥)

✅Verification – *"Are we building the product right?"*

- Documents, design, code check karna
- Mostly **development ke time**
- Example:
  - Design sahi hai?
  - Code requirement ke according likha?

👉 **No execution**, sirf checking.

✅Validation – *"Are we building the right product?"*

- Final software user ke kaam ka hai ya nahi
- Mostly **end me**
- Example:
    - User ka problem solve ho raha?
    - Customer satisfied hai?

👉 **Software run karke check karte hain**.

🧠 Testing

- Testing mainly **verification ka part** hota hai
- Bugs ko execute karke dhoondhna

🔷 3. Fault, Error, Failure, Bug (Confusion clear karo 😄)

Is course me **sab same maana jayega**, but concept samjho:

- **Fault / Bug / Defect**
  → Code me galti (wrong logic)
- **Error**
  → Software ke andar wrong state
- **Failure**
  → User ko galat output milna

👉 **Simple line**:
Code me galti → run karne par problem → user ko wrong result

Ek aur quick example 😄 (Calculator)

Fault → formula galat likh diya
Error → program wrong value compute karta hai
Failure → screen pe wrong answer show hota hai

🔷 4. Test Case kya hota hai?

Test Case = Input + Expected Output

Example:

```
Input: 2 + 3
Expected Output: 5
```

- Agar output = 5 → ✅Test Pass
- Agar output ≠ 5 → ❌Test Fail (Bug mila 🎉)

Test case me aur cheezein hoti hain:

- Test Case ID
- Steps
- Expected result

🔷 5. Types / Levels of Testing

🧩 1. Unit Testing

- Developer karta hai
- Small unit (function/method) test

- Coding ke time

🧩 2. Integration Testing

- Multiple modules ko jod ke test
- Example: Login + Database

🧩 3. System Testing

- Poora software ek saath
- Real environment me

🧩 4. Acceptance Testing

- Customer / End user karta hai
- Final approval ke liye

🧩 Beta Testing

- Real users ke paas release se pehle
- Feedback lene ke liye

🔷 6. Other Important Testing Types

- **Functional Testing** – Features kaam kar rahe?
- **Performance Testing** – Speed, response time
- **Stress Testing** – Load zyada ho to?

- **Usability Testing** – UI easy hai ya nahi
- **Regression Testing** – New change se purana code to nahi toota

---

🔷 7. Black Box vs White Box Testing

⚫ Black Box Testing

- Code ke andar kya hai → nahi dekhte
- Sirf input → output
- Tester ko code ka idea nahi hota

⚪ White Box Testing

- Code ke andar logic check
- Loops, conditions, paths
- Mostly developer karta hai

🔷 8. Test Activities (Testing ke Steps)

1️⃣ Test Design

- Sabse important step
- Effective test cases banana
- Mostly manual

2️⃣ Test Automation

- Test cases ko script me convert karna
- Tools use hote hain

3️⃣ Test Execution

- Tests run karna
- Result record karna

4️⃣ Test Evaluation

- Bugs analyse karna
- Root cause nikalna

🔷 9. Testing ki Need (Motivation 🔥)

Real life failures:

- 🚀 **Ariane-5 Rocket** explode – software error
- 🏥 **Therac-25** – patients ki death
- 💻 **Intel Pentium Bug** – millions ka loss

👉 Conclusion:

- Software error → **loss of money, life, trust**
- Isliye testing **mandatory** hai

level

- Level 0: There is *no* difference between *testing* and *de-bugging*.
- Level 1: The purpose of testing is to show *correctness*.
- Level 2: The purpose of testing is to show that software *doesn't work*.
- Level 3: The purpose of testing is not to prove anything specific, but to *reduce the risk* of using the software.
- Level 4: Testing is a *mental discipline* that helps all IT professionals develop higher quality software.

Level

🔷 10. SDLC (Software Development Life Cycle)

Phases:

1. Requirement & Planning
2. Design & Architecture
3. Development (Coding + Unit Testing)
4. Testing
5. Maintenance

👉 Best practice:
Testing har phase me honi chahiye, sirf end me nahi.

🔷 11. Agile Model

- Aaj kal sabse popular
- Small-small iterations

- Testing + development saath-saath

🔷 12. Automation & JUnit (Basic Idea)

- **JUnit** = Java testing framework
- Mostly **Unit Testing** ke liye
- Assertions use hote hain:
    - `assertEquals`
    - `assertTrue`
    - `fail`

Example:

```
assertEquals(5, add(2,3));
```

Agar output 5 nahi aaya → test fail.

🔚 Week-1 ka Final Summary 🧠

✔ Software testing = bugs dhoondhna
✔ Testing se software perfect prove nahi hota
✔ Testing SDLC ka important part hai
✔ Automation (JUnit) testing fast banata hai
✔ Real world me testing life-saving ho sakti hai

Week-2

Week-3

Week-4

Week-5

Week-5

📘 LOGIC BASICS – SOFTWARE TESTING (NOTES)

⭐1. Introduction to Logic

- Formal logic started around **320 BC** with the work of **Aristotle**.
- In software testing, logic helps in **designing test cases** and checking program correctness.
- Mainly used logic type:
  👉 **Predicate Logic (First-Order Logic)**

⭐2. Propositional Logic

- Deals with **propositions (statements)** that are either:
  - **True (T)** or
  - **False (F)**
- Complex statements are formed using **logical connectives**.

🔷 Common Logical Connectives

- OR → ∨∨∨ (Disjunction)
- AND → ∧∧∧ (Conjunction)
- NOT → ¬¬¬ (Negation)

- IMPLIES → →→→ or ⊃⊃⊃
- IFF (if and only if) → ↔↔↔ or ≡≡≡

⭐3. Atomic Propositions

- Basic building blocks of logic.
- A **declarative sentence** that is either true or false (not both).

✔ Examples

- New Delhi is the capital of India.
- 2 + 3 = 5.
- Today is Friday.

⭐4. Syntax of Propositional Logic

- Infinite set of propositions:
  P={p0,p1,…}P = \{p_0, p_1, …\}P={p0,p1,…}
- Rules:
  - Every proposition is a formula.
  - If ααα is formula → ¬α¬α¬α is also formula.
  - If α,βα, βα,β are formulas → α∨βα ∨ βα∨β is formula.

⭐5. Derived Logical Operators

- AND:
  👉 α∧β=¬(¬α∨¬β)α ∧ β = ¬(¬α ∨ ¬β)α∧β=¬(¬α∨¬β)

- IMPLIES:
  👉 α⊃β=¬α∨βα ⊃ β = ¬α ∨ βα⊃β=¬α∨β

- IFF:
  👉 α≡β=(α⊃β)∧(β⊃α)α ≡ β = (α ⊃ β) ∧ (β ⊃ α)α≡β=(α⊃β)∧(β⊃α)

⭐6. Semantics (Meaning of Formulas)

- A **valuation** assigns True or False to propositions.
- Rules:
  - v(¬β)=Tv(¬β)=Tv(¬β)=T if v(β)=Fv(β)=Fv(β)=F
  - v(β∨γ)=Tv(β ∨ γ)=Tv(β∨γ)=T if at least one is True

⭐7. Truth Tables

- Used to find truth value of logical formulas.
- Break formula into **sub-formulas** and evaluate step-by-step.

✔ Important Results

- p∧qp ∧ qp∧q → True only if both true
- p∨qp ∨ qp∨q → True if any one true
- p→qp → qp→q → False only when p = T and q = F
- p≡qp ≡ qp≡q → True when both same

⭐8. Satisfiability & Validity

- **Satisfiable:**
  Formula is true for **at least one valuation**.

- **Valid / Tautology:**
  Formula is true for **all valuations**.

✔ Examples

- p∨¬pp ∨ ¬pp∨¬p → Always true (Tautology)

- p∧¬pp ∧ ¬pp∧¬p → Always false (Not satisfiable)

⭐9. Satisfiability Problem

- To check satisfiability → construct **truth table**.

- Time complexity → **Exponential**.

- SAT problem in propositional logic is **NP-Complete**.

⭐10. Predicate Logic Basics

- Needed to reason about **program variables and data values**.

- A **predicate** → expression that evaluates to True/False.

- May contain:

  - Variables

  - Functions

  - Relational operators (<, >, ≤, ≥, =)

✔ Example Predicates

- x+y≤5x+y ≤ 5x+y≤5
- flag
- f(x)f(x)f(x)

⭐11. Clauses

- A **clause** = predicate **without logical operators**.
- Example:
  (x≥y)∨¬flag∨f(x)(x ≥ y) ∨ ¬flag ∨ f(x)(x≥y)∨¬flag∨f(x)

Clauses are:

- x≥yx ≥ yx≥y
- flag
- f(x)f(x)f(x)

⭐12. Predicate Logic Satisfiability

- Predicate is satisfiable if **some assignment of values** makes it True.
- General satisfiability problem in predicate logic is **undecidable**.
- Tools used:
  - **SAT solvers**
  - **SMT solvers**

📘 LOGIC COVERAGE CRITERIA – NOTES

⭐1. Logic Coverage Criteria (Overview)

Software testing me logical predicates ko test karne ke liye different **coverage criteria** use hote hain.

🔷 Main Types

- Predicate Coverage (PC)
- Clause Coverage (CC)
- Combinatorial Coverage
- Active Clause Coverage (ACC)
    - GACC (General Active Clause Coverage)
    - CACC (Correlated Active Clause Coverage)
    - RACC (Restricted Active Clause Coverage)
- Inactive Clause Coverage (ICC)
    - GICC
    - RICC

🔥 Quick Summary Table

Coverage	Idea
PC	P true & false
CC	Clar clause T & F
Combinatorial	All combinations
ACC	Clause affects P
GACC	Clause affects P (general)
CACC	Clause flip → P flip
RACC	Same minors + clause flip
ICC	Clause change → P same
GICC	ICC (loose)
RICC	ICC (strict)

⭐2. Test Cases for Coverage

- Predicate ya clause **valid / contradiction na ho** to easily test cases mil jate hain.
- Combinatorial coverage → **truth table ke sab combinations** test karne hote hain.
- SAT / SMT solver se **satisfying assignments** mil sakte hain.

⭐3. Correct Logical Predicate Banana

- Test case likhne se pehle **reachability problem** solve karna hota hai (RIPR model).
- Complex cases me **symbolic execution** ka use hota hai.

⭐4. Clause Determining Predicate (Very Important)

Agar predicate **p** me clause **c** ho:

- pc=truep_{c=true}pc=true → jab c ko true replace kare
- pc=falsep_{c=false}pc=false → jab c ko false replace kare

👉 Condition:

pc=pc=true⊕pc=falsep_c = p_{c=true} \oplus p_{c=false}pc=pc=true⊕pc=false

- Agar pc=truep_c = truepc=true → clause predicate ko **determine karta hai**
- Agar pc=falsep_c = falsepc=false → predicate clause se **independent hai**

⭐5. Examples

✔ Example 1

Predicate:

p=a∨bp = a \vee bp=a∨b

👉 For clause **a**

pa=¬bp_a = \neg bpa=¬b

Meaning:

- a predicate ko tab determine karega jab **b = false** ho.

✔ Example 2

Predicate:

p=a∧bp = a \wedge bp=a∧b

👉 For clause **a**

pa=bp_a = bpa=b

Meaning:

- a predicate ko tab determine karega jab **b = true** ho.

✔ Example 3

Predicate:

p=a≡bp = a \equiv bp=a≡b

👉

pa=truep_a = truepa=true

Meaning:

- a **har condition me predicate ko determine karega**.

⭐6. Infeasible Criteria (Important Theory)

- Agar pc=truep_c = truepc=true → ICC criteria infeasible.
- Agar pc=falsep_c = falsepc=false → ACC criteria infeasible.
- Matlab clause **redundant** hai.

⭐7. Redundant Clause Example

Predicate:

p=a∧b∨a∧¬bp = a \wedge b \vee a \wedge \neg bp=a∧b∨a∧¬b Simplify →

p=ap = ap=a

👉 Clause **b** predicate ko determine nahi kar sakta.

⭐8. More Example (Very Important for Exam)

Predicate:

p=a∧(b∨c)p = a \wedge (b \vee c)p=a∧(b∨c)

👉

- pa=b∨cp_a = b \vee cpa=b∨c
- pb=a∨¬cp_b = a \vee \neg cpb=a∨¬c
- pc=a∨¬bp_c = a \vee \neg bpc=a∨¬b

Meaning:

- a tab determine karega jab **(b∨c)=true** ho.

⭐9. TRs for ACC (Test Requirements)

Example predicate:

p=(a∨b)∧cp = (a \vee b) \wedge cp=(a∨b)∧c

📊 **Page 16 diagram** me truth table diya hai jisme 8 combinations dikhaye gaye hain (TTT, TTF, TFT … etc.) aur unke output values.
Is example me:

- pa=¬b∧cp_a = \neg b \wedge cpa=¬b∧c
- pb=¬a∧cp_b = \neg a \wedge cpb=¬a∧c
- pc=a∨bp_c = a \vee bpc=a∨b

⭐10. GACC Criteria

Rule:

- Major clause ko **true & false dono** lena hai.
- Minor clauses ko aise set kare ki major clause predicate ko determine kare.

📊 **Page 17 table** ke according

- Kuch test rows identical ho jati hain.
  👉 Final me **sirf 4 test cases enough hote hain** GACC satisfy karne ke liye.

⭐11. CACC vs GACC

- Exercise me diya hai → CACC ka result **GACC jaisa hi** hota hai.
- Practical testing me **CACC best mana jata hai**.

🐴 LOGIC COVERAGE – DONKEY LEVEL EXPLANATION

⭐Step-1: Sabse pehle LOGIC kya hota hai

Socho 😌

👉 Tumhare paas ek **switch system** hai

- a = light switch
- b = fan switch
- c = AC switch

Ab ek **rule (predicate)** hai:

👉 **Room cool hoga agar:**

AC ON ho OR fan ON ho Ye

pura rule = **predicate** Simple

👍

📌 Predicate = koi bhi rule / condition jo
TRUE ya FALSE ho sakta hai.

⭐Step-2: Clause kya hota hai

Predicate ke andar jo choti conditions hoti hain
wo **clauses** hoti hain.

Example:

p = a OR b

👉 clauses = a ,bBas

👍

📌 Clause = single condition
without AND / OR etc.

⭐Step-3: Testing me problem kya hai

Software tester ko check karna hota hai:

👉 kya system **har condition me sahi kaam kar raha hai**

Isliye different **coverage rules** banaye gaye.

🐴 TYPE-1 → Predicate Coverage (sabse easy)

Rule:

👉 Predicate ek baar TRUE hona chahiye
👉 Predicate ek baar FALSE hona chahiye Example:

p = a OR b

Test-1
a = T , b = F → p TRUE

Test-2
a = F , b = F → p FALSE

Ho gaya 👍

📌 Bas predicate ka output test karna.

🐴 TYPE-2 → Clause Coverage

Ab thoda smart tester 😎

Rule:

👉 Har clause ko

- ek baar TRUE

- ek baar FALSE

banana hai

Example:

p = a OR b

Test-1
 a = T , b = T

Test-2
 a = F , b = F

Ab:

- a bhi true/false
- b bhi true/false

Done 👍

📌 Clause coverage me predicate true false hona **zaroori nahi**.

🐴 TYPE-3 → Combinatorial Coverage (over smart 🤯)

Rule:

👉 **Saare combinations test karo**

a	b
T	T
T	F
F	T
F	F

Ye best hai
 but tests bahut zyada ho jate hain 😭

Isliye real life me mushkil.

🐴 Step-4: Smart idea — Active Clause Coverage

Tester sochta hai:

👉 Har clause ko aise test karo ki
wo predicate ko control kare

Matlab:

👉 clause change karo → predicate bhi change hona chahiye

Isko bolte hain:

⭐**Clause determines predicate**

⭐Example (very important)

p = a OR b

Case-1
a = T → p hamesha TRUE
👉 yaha b useless 😴

Case-2
a = F
👉 ab b decide karega p

Isliye test me
b ko tab test karo jab a = F

Ye hi logic coverage ka core idea hai 👍

🐴 Step-5: Active Clause Coverage (ACC)

Rule:

👉 Har clause ko
major clause banao
aur dekho kya wo predicate ko control karta hai

Example:

p = a OR b

Tests:

- a major → b = F
- b major → a = F

Total 3 tests enough 👍

🐴 Step-6: ACC ke 3 types

⭐GACC (General)

Minor clauses same ho ya different
koi tension nahi

Simple 👍

⭐CACC (Best wala ⭐)

Rule:

👉 Major clause TRUE → predicate TRUE
👉 Major clause FALSE → predicate FALSE Matlab

direct impact dikhe

Isliye real testing me ye best mana jata hai.

⭐RACC (Strict teacher 😡)

Rule:

👉 Minor clauses SAME hone chahiye
jab major true ya false ho

Bahut difficult
kabhi kabhi **possible hi nahi**.

🐴 Step-7: Inactive Clause Coverage (opposite idea)

Yaha tester check karta hai:

👉 clause change karo
👉 but predicate change **na ho**

Example:

Reactor system me
override mode me valve open/close
system behaviour same hona chahiye.

🧠 FINAL DONKEY SUMMARY

👉 Predicate = full condition
👉 Clause = small part

Coverage levels:

1️⃣ Predicate coverage
→ output true & false

2️⃣ Clause coverage
→ har clause true & false

3️⃣ Combinatorial
→ all combinations

4️⃣ ACC
→ clause predicate ko control kare

⭐ Best practical → **CACC**

🐴 LOGIC COVERAGE ON SOURCE CODE – DONKEY LEVEL

⭐ Step-1: Real program me predicate kahan se aata hai

Simple 😌

👉 Jab code me **if / while / for / switch** hota hai
to usme jo condition hoti hai = **predicate**

Example:

if (x > 10 && y < 5)

👉 Ye pura = predicate

📌 Testing me isi ko test karna hota hai.

⭐ Step-2: Source code testing mushkil kyu

Book bolti hai 2 main problem 😭

🔴 (1) Reachability

👉 Pehle us line tak pahuchna padega

Example:

if line 50 pe predicate hai
to test input aisa hona chahiye
ki program line-50 tak aaye

🔴 (2) Controllability

👉 Predicate me jo variables hain
wo direct input nahi hote

Matlab indirectly values set karni padti hain

📌 Isliye extra conditions lagani padti hain.

🐴 Step-3: Thermostat example story

Socho 😄

Ek **heater system** hai

Heater ON hoga agar:

👉 temperature bahut kam ho
👉 ya user override kare
👉 aur heater last run ke baad wait time complete hoActual

predicate simplified:

p = (a OR (b AND c)) AND d

where

- a → room temp bahut kam
- b → override ON
- c → override temp condition
- d → enough waiting time

📌 Ye book ne simplify karke diya hai.

⭐Step-4: Predicate coverage yahan kya karega

Rule simple:

👉 ek test jisme heater ON
👉 ek test jisme heater OFF

Example (page-9 idea):

- a = T
- b = T
- c = T
- d = T

→ heater ON

Ho gaya predicate true 👍

🐴 Step-5: Ab smart testing (CACC)

Goal:

👉 har clause ko aise test karo
ki **wo heater decision ko control kare**

Example:

Clause **a** test karna hai

To condition b,c,d aise set karo
ki:

👉 a TRUE → heater ON
👉 a FALSE → heater OFF

Isko bolte:

⭐**Clause determines predicate**

Book me formula nikala:

pa = (!b OR !c) AND d

Matlab:

👉 a tab important banega
jab d TRUE ho
aur b AND c dono TRUE na ho

Ye logic page-11 me derive kiya hai.

⭐Step-6: Final CACC test idea

Table (page-12 concept):

Har clause ke liye:

- ek test TRUE
- ek test FALSE

But predicate output change hona chahiye

Result:

👉 total **6 tests enough**

Kyunki kuch rows duplicate hoti hain.

⭐Step-7: Real inputs kaise banaye

Book ne bataya:

Clause **a TRUE** banane ke liye:

👉 curTemp = 63
👉 dTemp = 69
👉 thresholdDiff = 5

Clause **a FALSE** banane ke liye:

👉 curTemp = 66Similarly:

- b → override true/false
- c → overTemp values
- d → waiting time values

Is tarah tester real numbers set karta hai 👍

🧠 FINAL DONKEY SUMMARY

👉 Code me predicate = if condition

👉 Testing me:

1️⃣ Line tak pahuchna = reachability
Variables control karna = controllability
2️⃣
3 Predicate coverage
❗
→ heater ON & OFF

4️⃣ CACC
→ har clause ko aise test karo
ki heater decision change ho

👉 Thermostat example me
⭐ **6 tests enough**

✅ **PC (Predicate Coverage)**
→ Ensure the **entire predicate evaluates to TRUE at least once and FALSE at least once.**

✅ **CC (Clause Coverage)**
→ Ensure **each individual clause in the predicate evaluates to TRUE and FALSE at least once.**

✅ **CoC (Combinatorial Coverage)**
→ Ensure **all possible combinations of truth values of the clauses are tested.**

✅ **ACC (Active Clause Coverage)**
→ Ensure **each clause independently affects (determines) the predicate outcome.**

✅ **GACC (General Active Clause Coverage)**
→ Ensure **a major clause determines the predicate, while minor clause values may vary.**

✅ **CACC (Correlated Active Clause Coverage)** ⭐
→ Ensure **changing the major clause alone changes the predicate outcome (TRUE ↔ FALSE).**

✅ **RACC (Restricted Active Clause Coverage)**
→ Ensure **the major clause determines the predicate while minor clauses remain fixed.**

✅ **ICC (Inactive Clause Coverage)**
→ Ensure **the predicate outcome remains unchanged even when the clause value changes.**

📊 ACC Types Comparison Table

Feature	ACC (General Idea)	GACC	CACC ⭐	RACC
Main goal	Clause should control predicate	Clause should control predicate	Clause should control predicate	Clause should control predicate

Week-6

Week-6

Week-7

Week-7

📘 1. Symbolic Execution kya hota hai?

👉 Simple language me:

Symbolic Execution = Program ko actual numbers ke saath nahi, balki symbols (variables) ke saath run karna

📌 Example:

- Normal execution: `a=1, b=3`
- Symbolic execution: `a = α1, b = α2`

💡 Goal:
➡Ye pata lagana ki **kaunse inputs pe program ka kaunsa part run hoga**

📘 2. Normal Execution vs Symbolic Execution

🔷 (Page 4-6 example)

Program:

x = a + b
y = b + c
z = x + y - b

🔶 Normal Execution (Page 5 table)

Input: `a=1, b=3, c=5`

👉 Step-wise:

- x = 1+3 = 4
- y = 3+5 = 8
- z = 4+8-3 = 9

👉 Final output = **9**

📌 Table me:

- ? = undefined
- – = unchanged

🔶 Symbolic Execution (Page 6 table)

👉 Inputs:

- a = α1
- b = α2
- c = α3

👉 Steps:

- x = α1 + α2
- y = α2 + α3
- z = α1 + α2 + α3

👉 Final:
✔Output = **α1 + α2 + α3**

💡 Meaning:
➡Ek hi run me humne **general formula** nikal liya

📘 3. Path Condition (PC) kya hota hai?

👉 Very important concept 🔥

PC = conditions jo satisfy honi chahiye kisi path ko follow karne ke liye

📌 Initially:

PC = true

📌 Example:

if (x > 10)

👉 Then branch:

PC = PC ∧ (x > 10)

👉 Else branch:

PC = PC ∧ (x ≤ 10)

💡 Symbolic execution me:
➡**Dono branches explore hoti hain** (normal execution me sirf ek)

📘 4. Example 2 (Error Finding) 🔥

(Page 7–9)

Code idea:

```
z = 2*y
if(z == x)
  if(x > y+10)
    ERROR
```

🔶 Symbolic state:

- x = x0
- y = y0
- z = 2*y0

🔶 Path Conditions:

Path 1:

z == x → 2*y0 == x0

Path 2:

x > y+10

👉 Combine:

(2*y0 == x0) ∧ (x0 > y0 + 10)

🔶 Solve karke:

👉 Inputs milte hain:

- (x=30, y=15)

➡Ye input **ERROR tak le jaata hai**

💡 Insight:
👉 Symbolic execution se hum **bug find kar sakte hain automatically**

📘 5. Execution Tree kya hota hai?

(Page 9)

👉 Program ke saare possible paths ko tree form me dikhate hain

```
      start

     /    \
   true    false
   /         \
 next        next
```

💡 Har branch = ek condition ka result

📘 6. Formal Methods kya hote hain?

(Page 10-11)

👉 Mathematical techniques use karke program ko analyze karna

Types:

1. Model checking
2. Theorem proving
3. Program analysis

🔶 Testing vs Proving

Testing	Proving
Sample inputs	All inputs
Fast	Difficult
Not complete	Complete guarantee

💡 Symbolic execution = **in dono ke beech ka approach**

📘 7. Symbolic State (σ) kya hota hai?

(Page 14-16)

👉 Mapping:

σ = { variable → symbolic expression }

📌 Example: x

→ x0

y → y0
z → 2*y0

🔶 Rules:

1. Start:

σ = empty

2. Input:

x = sym_input → x → x0

3. Assignment:

z = 2*y → z → 2*y0

📘 8. Path Constraint (PC) ka Working

(Page 17)

👉 Jab IF aata hai:

if(e)

➡2 paths bante hain:

1. PC ∧ e
2. PC ∧ ¬e

💡 Important:

- Agar PC satisfiable nahi hai → path terminate

📘 9. Test Case Generation kaise hota hai?

(Page 18)

👉 End me:

- PC solve karo (using constraint solver)
- Concrete inputs milte hain

👉 Ye inputs:
✔same path follow karenge
✔testing coverage badhayenge

📘 10. Loops ka problem 😵

(Page 21-22)

👉 Example:

while(N > 0)

💡 Problem:
➡Infinite paths possible

Example:

- N1 > 0
- N2 > 0
- ...
- Nn ≤ 0

👉 Final:

sum = N1 + N2 + ... + Nn

📘 11. Limitations of Symbolic Execution ❌

(Page 24-25)

1. PC solve karna mushkil (NP-complete 😵)
2. External functions ka code nahi hota
3. Too many paths (path explosion 🚨)

📘 12. Modern Improvements

(Page 26)

👉 SMT solvers:

- fast
- powerful
- large constraints handle karte hain

📘 13. Concolic Testing 🔥 (Most Important)

👉 Full form:
Concolic = Concrete + Symbolic

(Page 27)

🔶 Idea:

👉 Program ko:

- normal (concrete) + symbolic dono se run karo

🔶 Steps:

1. Random input lo
2. Program run karo
3. PC collect karo
4. PC ko modify (negate) karo
5. New input generate karo

🔶 Example (Page 28)

1st run:

x=22, y=7

➡PC:

$x0 \neq 2*y0$

Next:

solve: $x0 = 2*y0 \rightarrow x=2, y=1$

Next:

solve: $(x0 = 2*y0) \wedge (x0 > y0 + 10)$
$\rightarrow x=30, y=15$

➡ERROR reached 🎯

📘 14. Concolic ka advantage 💯

✔Random + smart search
✔Faster path exploration
✔Real-world programs me useful

📘 15. Real Tools

(Page 32)

👉 Popular tools:

- CUTE
- jCUTE
- PEX
- SAGE

🧠 Final Summary (Exam ke liye)

👉 Symbolic Execution:

- inputs = symbols
- output = expressions
- PC = constraints

👉 Concolic:

- concrete + symbolic
- smart testing
- faster bug detection

📘 1. DART kya hai? (Directed Automated Random Testing)

👉 Simple definition:

DART = Random Testing + Symbolic Execution (Concolic)

📌 Matlab:

- Random inputs bhi use karta hai
- Aur smart tarike se naye inputs generate karta hai

💡 Isliye:

➡Ye ek **concolic testing technique** hai

📘 2. DART ka main goal 🎯

👉 Automatically:

- test cases generate karna
- program ke **different paths explore karna**
- bugs find karna

📘 3. DART ka core idea 🔥

(Page 3)

DART 3 cheeze combine karta hai:

🔷 **1. Interface extraction**

👉 Program ke inputs kaha se aa rahe hain → identify karta hai

---

🔷 **2. Random Testing**

👉 Random inputs generate karta hai

🔷 **3. Directed Search**

👉 Symbolic constraints use karke:
➡next input generate karta hai jo **naya path explore kare**

📘 4. DART ka advantage 💯

(Page 4)

✔Test driver khud banata hai
✔Fully automatic testing
✔Detect kar sakta hai:

- crash
- assertion fail
- infinite loop

📘 5. Example samjho (VERY IMPORTANT 🔥)

(Page 5-9)

Code:

```
if (x != y)

  if (2*x == x+10)

     abort();
```

🔶 **Problem:**

👉 Random testing se error milna mushkil hai

💡 Kyun?

➡Condition rare hai:

$2*x = x + 10 \rightarrow x = 10$

🔶 **DART ka approach:**

Step 1: Random input

$x = 354, y = 34567$

➡First condition true

➡Second false → no error

Step 2: Symbolic constraints banate hain

$x0 \neq y0$

$2*x0 \neq x0 + 10$

Step 3: Negate constraint

$2*x0 = x0 + 10$

👉 Solve: x

= 10

Step 4: New run

x = 10, y = 4566

➡ ERROR hit 🎯

💡 Insight:
👉 DART **random se start karta hai → phir smart ho jata hai**

📘 6. Directed Search ka idea 🔁

(Page 8)

👉 Har run ke baad:

- path constraint collect karo
- usko modify karo
- new input generate karo

➡Repeat till:
✔saare paths explore ho jaye

📘 7. Execution Model (Deep concept ⚠)

(Page 11)

👉 DART:

- program ko **normal + symbolic dono tarike se run karta hai**

👉 Iske liye:
➡program ko **instrument (modify)** kiya jata hai

📘 8. Symbolic Variables (Memory level) 🧠

(Page 12)

👉 Variables ko memory address se represent karte hain:

m → symbolic variable

👉 Expression ho sakta hai:

- addition

- multiplication
- comparison
- pointer

📘 9. Program Execution Tree 🌳

(Page 17)

👉 Program = tree

- Assignment → 1 path
- Condition → 2 paths

👉 Har input:
➡ek path choose karta hai

📘 10. Symbolic Memory (S) kya hota hai?

(Page 18)

👉 Mapping:

S = { address → symbolic expression }

👉 Start:

S(m) = m

📘 11. Symbolic Evaluation (Important ⚠️)

(Page 19-21)

👉 Jab expression evaluate hota hai

Case:

1. Simple → symbolic form
2. Complex (non-linear) → concrete use

🔴 **2 Important Flags:**

1. **all linear = 0**
   - jab expression complex ho jaye
2. **all locs definite = 0**
   - jab pointer/value unknown ho

💡 Matlab:
➡DART kabhi-kabhi symbolic छोड़कर concrete use karta hai

📘 12. Test Driver (Heart of DART) ❤

(Page 22-23)

👉 Structure:

repeat:

  random testing

  directed search

until all paths explored

👉 Agar:

- exception aaye → BUG FOUND
- sab paths cover → DONE

📘 13. Instrumented Program kya karta hai?

(Page 24-27)

👉 Ye program:

- original code run karta hai

- saath me symbolic tracking bhi karta hai

🔶 **Assignment case:**

m = e

👉 Update:

- symbolic memory
- concrete memory

🔶 **IF condition:**

if(e)

👉 Update:

- PC add hota hai
- branch record hota hai

📘 14. Stack concept 🧠 (Very Important)

(Page 30)

👉 Stack store karta hai:

(branch, done)

Meaning:

- branch = 0/1 (false/true)
- done = explored or not

💡 Use:
➡Next run me pata lagta hai:

- kaunsa path explore ho chuka
- kaunsa baaki hai

📘 15. Solve Path Constraint 🔍

(Page 29)

👉 Process:

1. Last unexplored condition find karo
2. usko negate karo
3. solve karo
4. new input generate karo

📘 16. Theorem (Exam me aa sakta hai 🔥)

(Page 33)

👉 3 cases:

1. "Bug found" → definitely bug hai
2. Normal end → no bug + all paths covered
3. Otherwise → infinite run

📘 17. DART for C (Practical part)

(Page 34)

3 parts:

1. Interface extraction
2. Random driver
3. Directed search

📘 18. Interface Extraction (Important)

(Page 36)

👉 Identify inputs from:

- external variables
- external functions
- function arguments

📘 19. Random Test Driver

(Page 39-40)

👉 Example:

for(i=0; i<depth; i++)

random input → function call

👉 Ye simulate karta hai:
➡real environment

📘 20. random_init() function 🎲

(Page 41-43)

👉 Har type ke liye random value:

Case:

- int → random bits
- pointer → NULL ya malloc
- struct → recursively fill
- array → loop

💡 Important:
➡Ye realistic testing karta hai

📘 21. Directed Search Implementation

(Page 45)

👉 Tools:

- CIL (code analysis)
- Constraint solver (IP solve)

🧠 Final Summary (Exam ke liye 🔥)

👉 DART =
✔Random testing
✔Symbolic execution
✔Automatic test generation

👉 Flow:

1. Random input
2. Execute
3. Path constraint collect
4. Negate
5. Solve
6. New input

👉 Key terms:

- Path Constraint
- Stack (branch, done)
- Symbolic memory
- Directed search

📎 Source

Ye explanation aapke DART PDF se bana h

Week-8

Week-8

📘 1. SDLC kya hota hai?

👉 **SDLC = Software Development Life Cycle**

👉 Matlab:
Software banane ka pura process (start se end tak)

🔷 Steps:

1. Planning + Requirements
2. Design
3. Coding
4. Testing
5. Maintenance

📌 Simple:
➡Idea → Plan → Build → Test → Maintain

📘 2. Requirement kya hoti hai?

👉 IEEE ke according:

Requirement = jo system ko karna chahiye / jo user chahta hai

🔷 3 meanings:

1. User ki need
2. System ko satisfy karna hoga
3. Document form me likha hua

📌 Simple:
➡Requirement = “system kya karega”

📘 3. Requirement ke types (Level 1)

🔷 1. Product Requirements

👉 Product ke features

📌 Example:

- Phone ka color
- Window 1 sec me rukna

🔷 2. Process Requirements

👉 Development process se related

📌 Example:

- Safety rules follow karna

💡 Difference:

- Product → kya banega
- Process → kaise banega

📘 4. Requirement ke types (Level 2- MOST IMPORTANT 🔥)

🔷 1. Business Requirements

👉 Stakeholder ka view

📌 Example:

- Battery 24 hours chale

📌 Document:
👉 StRS (Stakeholder Requirement Spec)

🔷 2. User Requirements

👉 User kya chahta hai

📌 Example:

- App Hindi me ho

📌 Document:
👉 URS

💡 Ye ek contract jaisa hota hai

🔷 3. Functional Requirements 🔥

👉 System kya karega (behavior)

📌 Example:

- Ticket booking
- Payment system

👉 Input → Output define karta hai

📌 Document:
👉 FRS

🔷 4. Non-Functional Requirements 🔥

👉 System kaise kaam karega

📌 Example:

- 1 sec response
- 10,000 users handle

💡 Shortcut:

- Functional → WHAT
- Non-functional → HOW

🔷 5. Regulatory Requirements

👉 Laws/rules follow karna

📌 Example:

- ISO standards
- Country-specific rules

📘 5. Functional vs Non-functional (Important 🔥)

Functional	Non-functional
Kya karega	Kaise karega
Feature	Quality
Booking	Speed

📘 6. Black-box Testing kya hai?

👉 Testing without seeing code

👉 Sirf:

- Input
- Output

📌 Based on:

➡Requirements

💡 Matlab:
➡“andar kya ho raha hai → ignore”

📘 7. SDLC ka importance

👉 Kyun use karte hain?

✔Structured process
✔High quality software
✔Proper planning

🧠 Final Summary (Exam ke liye 🔥)

👉 SDLC:
➡Plan → Design → Code → Test → Maintain

👉 Requirement:
➡System kya karega

👉 Types:

- Business → stakeholder
- User → user need
- Functional → WHAT
- Non-functional → HOW
- Regulatory → rules

👉 Black-box:
➡Requirement based testing

📘 1. Functional Testing kya hota hai?

👉 Simple definition:

Functional Testing = program ko ek function ki tarah treat karna

📌 Matlab:
➡Input doge → Output milega y

= P(x)

💡 Focus:

- kya input diya
- kya output aaya

👉 andar kya ho raha hai → **ignore**

➡Ye ek **black-box testing technique** hai

📘 2. Black-box vs White-box (recap)

Black-box	White-box
Input-output based	Code based
No internal knowledge	Internal logic pata
Functional testing	Graph / logic coverage

📘 3. Functional Testing ka process (VERY IMPORTANT 🔥)

(Page 4)

🔷 Step 1:

👉 Input-output domain identify karo

🔷 Step 2:

👉 Important values choose karo

🔷 **Step 3:**

👉 Combination test karo

🔷 **Step 4:**

👉 Special output cases test karo

💡 Simple:
➡"Smart inputs choose karo jo bugs nikaale"

📘 4. Context ka importance (hidden concept ⚠️)

(Page 5-6)

👉 Kabhi-kabhi:
➡thoda context pata hona zaroori hai

📌 Example:

if (x ≥ 20)

👉 Agar tumhe ye nahi pata:
➡tum random values test karoge

👉 Lekin actual:

- x = 20
- x > 20
- x < 20

👉 Ye hi important test cases hain

💡 Insight:
➡"Blind testing = inefficient"

📘 5. Types of Functional Testing 🔥

(Page 7)

1. Equivalence Partitioning
2. Boundary Value Analysis
3. Decision Tables
4. Random Testing

📘 6. Equivalence Class Partitioning (ECP) 🔥

👉 Idea:

Similar inputs ko group me divide karo

👉 Har group se ek test case enough hai

🔶 Example (Tax system)

(Page 9-10)

Ranges:

- 1–29500
- 29501–58500
- 58501–100B
- invalid (<1)
- invalid (>limit)

👉 Total = 5 classes

👉 Test cases = 5

💡 Insight:
➡"Har value test karne ki zaroorat nahi"

📘 7. Boundary Value Analysis (BVA) 🔥

👉 Idea:

Boundary ke paas errors zyada hote hain

🔶 Example:

Range:

$1 \leq x \leq 100$

👉 Test:

- 0
- 1
- 2
- 99
- 100
- 101

💡 Rule:
➡Boundary ±1 test karo

🔶 Important Guidelines (Page 12)

1. Range → boundary points
2. Min/max → extreme values
3. Ordered set → first & last

💡 Insight:
➡"Programmer mistakes mostly boundaries pe hoti hain"

📘 8. Decision Tables 🔥 (Very Important)

👉 Problem:
➡Multiple inputs → complex conditions

👉 Solution:
➡Table bana lo

🔶 Concept:

- Conditions (C1, C2, C3)
- Effects (E1, E2, E3)
- Rules (R1–R8 combinations)

🔶 Example (Page 17 image)

👉 Conditions:

- Loan amount mentioned?
- Terms mentioned?

👉 Outputs:

- Process loan
- Process term
- Error

📌 Example:

- Dono YES → process both

- Dono NO → error

💡 Insight:
➡"All combinations systematically cover hote hain"

📘 9. Random Testing 🎲

👉 Idea:

Random inputs generate karo

🔶 Example (sqrt)

👉 Random numbers lo
👉 output verify karo

💡 Advantage:
✔Easy
✔Fast

💡 Disadvantage:
❌Rare bugs miss ho sakte hain

📘 10. Important Comparison 🔥

Technique	Idea
ECP	groups
BVA	boundaries
Decision Table	combinations
Random	guess

🧠 Final Summary (Exam ke liye 🔥)

👉 Functional Testing:
➡Input → Output based

👉 Key techniques:

- ECP → group
- BVA → edge
- Decision table → combo
- Random → guess

👉 Important:
➡Boundary errors sabse common

📎 Source

Ye explanation tumhare PDF se bana hai:

📘 1. ISP (Input Space Partitioning) kya hota hai?

👉 Simple idea:

Input Space Partitioning = Inputs ko groups (partitions) me divide karna testing ke liye

📌 Matlab:
➡Har possible input ko test nahi karte
➡Usko **groups me tod dete hain**

📘 2. Partition kya hota hai?

(Page 2)

👉 Partition = ek set ko parts me divide karna

Conditions:

1. Overlap nahi hona chahiye ❌
2. Sab cover hona chahiye ✅

S = S1 + S2 + S3

💡 Simple:
➡"Divide karo, lekin duplicate ya missing nahi hona chahiye"

📘 3. Testing me ISP ka use 🔥

(Page 3)

👉 Input domain ko partition karte hain

👉 Har partition ka behaviour same hota hai

💡 Isliye:
➡ek partition se ek test case enough

👉 Isi ko equivalence partitioning bhi bolte hain

📘 4. Characteristics kya hoti hain?

(Page 4)

👉 Partition banane ka base

📌 Example:

- input null hai ya nahi
- file sorted hai ya nahi

👉 Har characteristic → ek partition

📘 5. Valid Partition ke rules ⚠️

(Page 5)

1. **Completeness** → sab cover hona chahiye
2. **Disjoint** → overlap nahi hona chahiye

❌Wrong Example:

- sorted
- reverse sorted
- random

👉 Problem:
➡empty file sab me aa sakta hai

📘 6. Example (Tax system)

(Page 6-7)

👉 AGI ranges:

- 1–29500
- 29501–58500
- 58501–100B
- invalid (<1)
- invalid (>limit)

👉 Total partitions = 5

👉 Test cases = 5

💡 Insight:
➡“Har group se ek value lo”

📘 7. Input Domain Modelling 🔥

(Page 8)

👉 Steps:

1. Function identify karo
2. Inputs identify karo
3. Characteristics define karo
4. Test cases generate karo

👉 Test input = tuple

(x1, x2, x3)

📘 8. 2 Approaches 🔥

(Page 9)

🔷 1. Interface-Based

👉 Inputs ko alag-alag treat karo

✔Easy
❌Combinations miss ho sakte hain

🔷 2. Functionality-Based

👉 System ke behaviour ke basis par

✔Better coverage
❌Difficult

💡 Shortcut:

- Interface → simple
- Functionality → powerful

📘 9. Example (list search)

(Page 12-14)
Function:

findElement(list, element)

Interface-based:

- list null?
- list empty?

Functionality-based:

- element kitni baar aata hai
- first position pe hai ya nahi

💡 Insight:
➡Functionality approach deeper hai

📘 10. Partition choose kaise kare?

(Page 15-16)

👉 Balance important hai

- zyada partitions → better testing
- kam partitions → fast testing

🔷 Include:

✔valid values
✔invalid values
✔boundary values

❌Avoid:

❌overlapping
❌missing partitions

📘 11. Multiple Inputs problem 😵

(Page 17)

👉 Agar 2 inputs hain:

x → 3 partitions
y → 2 partitions

👉 Total combinations = 6

➡Inputs badhenge → combinations explode

📘 12. Coverage Criteria 🔥🔥 (MOST IMPORTANT)

(Page 18)

🔷 1. ACoC (All Combinations)

👉 Sab combinations test karo

💡 Example:

[A,B], [1,2], [x,y]

→ 2×2×2 = 8 tests

✔Best coverage
❌Too many tests

🔷 2. ECC (Each Choice)

👉 Har partition se ek value lo

✔Minimum tests
❌Weak

🔷 3. Pair-Wise Coverage (PWC) 🔥

👉 Har pair of values cover hona chahiye

✔Practical
✔Most used

💡 Insight:
➡"2 inputs ke relation pakdo"

🔷 4. T-wise Coverage

👉 Generalization of pair-wise

- t=2 → pairwise
- t=3 → 3-wise

✔Strong
❌Expensive

🔷 5. Base Choice Coverage (BCC)

👉 Ek base value choose karo

👉 baaki values ek-ek karke change karo

📌 Example:
Base = (A,1,x)

Tests:

- (B,1,x)
- (A,2,x)
- (A,1,y)

🔷 6. Multiple Base Choice (MBCC)

👉 Multiple base values use karo

✔Flexible
✔Better coverage

📘 13. Important Insight 🔥

(Page 25)

👉 Difference:

Method	Idea
ACoC	all
ECC	each
PWC	pairs

BCC	base

📘 14. Constraints (Very Important ⚠️)

(Page 32-33)

👉 Har combination possible nahi hota

📌 Example:

- list empty
- element found

👉 Impossible combination ❌

🔷 Types:

1. Cannot combine
2. Must combine

💡 Handling:

- invalid combinations remove karo

🧠 Final Summary (Exam ke liye 🔥)

👉 ISP:
➡Inputs ko groups me todna

👉 Rules:

- complete
- disjoint

👉 Approaches:

- Interface

- Functionality

👉 Coverage:

- ACoC → all
- ECC → each
- PWC → pair
- BCC → base

👉 Important:
➡Pair-wise sabse practical

📎 Source

Ye explanation tumhare PDF se bana hai: